

[F] Comparative Study of VoxNet and PointNet for 3D Object Classification, with an Autoencoder Extension of VoxNet

Eric Blohm
(Dated: June 4, 2025)

This project investigates the impact of input data structure on the performance of 3D classification models. I compare PointNet, which directly processes raw point clouds, to VoxNet, which operates on voxelized input requiring additional preprocessing. PointNet achieved a higher classification accuracy (90.5% vs. 86.9%), likely due to its ability to handle unstructured data. Additionally, I extend VoxNet with a decoder head to reconstruct partial inputs using an autoencoder architecture. The improved model achieved a 68.6% IoU, up from 56.2% without skip connections. Overall, the project shows that choices in input representation and model architecture have a significant impact on classification accuracy and reconstruction quality in 3D deep learning.

Project Topic: 3D Shape Classification and Reconstruction using Deep Learning

Teaching Assistant: Fredrik Skärberg

I. INTRODUCTION

Autonomous driving is one of the most widely discussed topics in modern technology, and the development of fully autonomous driving are accelerating at a rate never seen before. These vehicles can navigate with little to no human intervention, relying on sensors such as LiDARS and cameras to perceive their surroundings. This enables them to detect pedestrians, other vehicles and traffic lights in order to operate safely [1].

There are many advantages in autonomous driving, including less traffic, fewer accidents and better fuel economy. Furthermore, wide spread use of autonomous vehicles can reduce up to 90% of the accidents caused by humans, and 60% of the traffic congestion [1].

One of the core pillars of safe autonomous driving is the use of sensors to perceive the environment. To do this, 3D object detection is a crucial capability, since it allows the vehicles to identify and categorize objects. This method creates a bounding box around the object, and assigns the object with various information such as position and a class label [1].

This project will focus on the 3D classification component of object detection, which is responsible for assigning class labels to detected objects. There are three main approaches to 3D classification: Multi-View, Volumetric and Point- based methods. The multi-view approaches projects the 3D shape into multiple views and extract view-wise features, which are then fused together for accurate classification. Volumetric methods usually voxelize point clouds or meshes into a 3D grid (volume), and then applies 3D convolutions to the volumetric representation for classification. In contrast, point-based methods can operate directly on raw points clouds using architectures based on MLPs, CNNs or graph structures, without requiring voxelization or mesh conversion [2].

In addition to exploring classification, this project also investigates the potential of repurposing a classification network for 3D object reconstruction using an autoencoder architecture. As highlighted by recent research, 3D

reconstruction significantly advances robotics by bridging simple perception and sophisticated interaction with the environment, enabling robots to build coherent models of their surroundings essential for decision-making processes [3].

II. OVERVIEW

This section presents the key 3D deep learning classification methods relevant to the project.

VoxNet is a deep learning model designed to work on volumetric input such as voxel grids, where a voxel is either occupied (1) or free (0). It benefits from the simple design and the use of dense data, allowing 3D CNNs to be effectively utilized. The voxel grid representation can also be extended to encode probabilities, where a 1 indicate a high occupancy, and a 0 indicate a low occupancy. The probabilistic representation provides more information than a point cloud which only captures the positions of occupied points in space [4].

However, the process of converting to a volumetric representation from point clouds or meshes is computationally heavy. In addition, VoxNet is limited in its resolution, it utilizes a 32^3 volume, while higher resolution could capture more information. Finally, VoxNet was among the first models to work on real time classification of voxel grids, and their results outperformed other state-of-the-art models at the time. For instance ShapeNet, which has 12.4 million parameters, achieved lower accuracy on most of the tests, while VoxNet, with fewer than 1 million parameters performed better [4].

PointNet on the other hand works directly with raw point clouds as input, eliminating the computation overhead caused by the volumetric conversion. In fact, the PointNet was the first model to handle the unordered point clouds immediately as input. The key to their approach is utilizing a symmetric function such as max pooling. Effectively, the network learns to select interesting or informative points of the point clouds and encode the reason for their selection. Conversely, max pooling

TABLE I. **Architectures for object classification and reconstruction.** This table summarizes various methods used for 3D object classification and reconstruction, highlighting their key features and relevance to the project.

Method	Use case scenario	Features	Suitable for the project?
VoxNet	3D object classification.	Pros: compact and efficient architecture for real time classification. Cons: limited in its resolution and computational overhead.	Strong baseline model for voxel-based classification. Simple to implement and well-suited for comparisons.
PointNet	3D object classification and segmentation.	Pros: Works directly with point clouds instead of voxel grids. Cons: treats each point independently, thus lacks spatial awareness.	Highly relevant for comparing point-based representations vs. voxel-based.
3D Autoencoder	3D object reconstruction.	Pros: uses encoder-decoder structure, straightforward to attach a decoder head to an existing classifier. Cons: limited to reconstruction only, can not generate new objects.	Suitable for evaluating shape reconstruction. Complements classification models by exploring whether a decoder head can be attached to a classifier.

also acts as a bottleneck where all information flows, where it compresses the 3 dimensions (x,y,z) in to one dimension of n points. Another limitation is that the use of raw point clouds as input treats each point independently and does not capture local structures or the spatial relationship between points [5]. The limitations of PointNet motivated the development of PointNet++, which groups nearby points into local neighborhoods and applies PointNet recursively on these subsets to better capture local geometric features [6].

A **3D autoencoder** is used to complete the objects from partial input data. Due to its encoder-decoder structure it is straightforward to extend an existing classifier with a decoder head and start experimenting with parameters and model design [7]. However, a basic autoencoder structure is limited to reconstruction only, and lacks the ability to generate new objects.

Several 3D object classifiers outperform both VoxNet and PointNet. For instance *Dynamic Graph CNN (DGCNN)* which creates a local graph around the points and then applies edge convolutions to capture local geometries. Addressing the same limitations as PointNet++. However, the DGCNN has a high computational cost due to repeated nearest neighbour graph construction and search operations at each layer [8].

I chose not to proceed with Pointnet++ or DGCNN because my focus was to compare two models first of their kind, PointNet and Voxnet, which operate on different input representations. This comparison enables a clearer understanding of how input structure could influence model performance and size. Additionally, the goal was to explore whether a decoder head can be attached to a classifier, and thus create an efficient autoencoder for object reconstruction. A summary of the proposed methods and their respective advantages and disadvantages is provided in Table I.

III. METHOD

To implement VoxNet proposed in [4], the CAD files from ModelNet10 had to be preprocessed such that each object can be fed to the neural network. This preprocessing is called voxelization, and for VoxNet, it is the process of converting a point cloud or mesh to a volume made of 32^3 voxels. A voxel is a cube and can be thought of as the pixel counterpart in 3D. Lastly, to reduce the training time, the voxel grids were precomputed and saved in separate files.

The network architecture used in this project is VoxNet and is made of two 3D convolutional blocks with relu activation and dropout after each block [4]. Moreover, max pooling followed by a dense layer is incorporated for classification. Figure 1 show the detailed architecture.

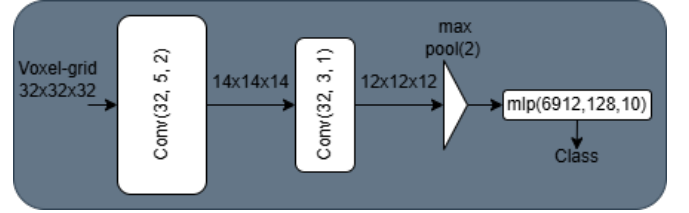


FIG. 1. **VoxNet architecture.** $Conv(f,k,s)$ represent a convolutional layer with f number of filters, a kernel size of k and a stride length of s . Moreover, $max\ pool(m)$ describes pooling with area m . Finally, mlp describes a multilayer perceptron.

Implementing a point cloud based architecture allows for comparing different approaches to object classification. The classification head of PointNet benefits from its simple yet effective architecture, see Figure 2. In practice, the initial part of the network does not use traditional multilayer perceptrons (MLPs) applied independently to each point. Instead, since weights are shared across all points and the input has three channels corresponding to the (x, y, z) coordinates, 1D convolutions are used to efficiently implement these shared MLPs. The classifier is composed of fully connected layers arranged

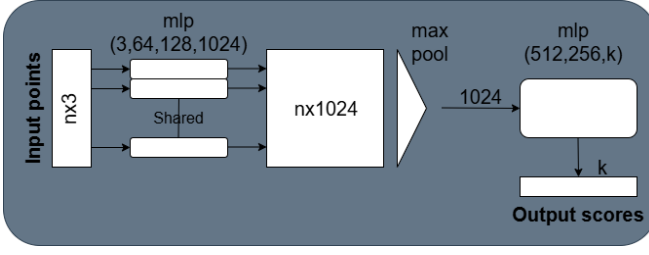


FIG. 2. **PointNet for classification.** This version is without input transformations, based on the original PointNet architecture [5]. It performs several 1D convolutions, into a max pooling and then downsamples and classifies an object. Batch normalization and relu is applied after every layer, both convolutional and linear.

as $MLP(1024, 512, 256, k)$, where k is the number of output classes. Moreover, batch normalization is applied after every block, and dropout before the final classification layer to reduce overfitting. Finally, the preprocessing includes uniformly random sampling of point from each object after loading it as a mesh. The sampled point cloud is then centered around its mean, and scaled to the unit sphere.

For both models, holdout validation combined with model checkpointing was incorporated to reduce the risk of using an overfitted model and to use the best possible state when testing. Model checkpointing refers to saving the state of the model when a new lowest validation loss is seen. The best model is then used for testing.

Finally, VoxNet was converted to an autoencoder for reconstruction of objects. To train for reconstruction, the input was sliced 50% across all dimensions in an attempt to mimic a point of view. Furthermore, the VoxNet architecture was kept in its entirety acting as an encoder, and the latent space was increased from 128 to 512. Moreover, deconvolutional blocks was added to upsample from the latent space, these blocks matched the size of the encoder, acting as the decoder. To finalize the model, concatenation skip connections was incorporated to precisely reconstruct the input, while having the ability to fill in the missing pieces of the input. In addition, batch normalization was added to stabilize training. Lastly, considering that a voxel is either 0 or 1, binary cross-entropy was used as the loss function. The final autoencoder structure is shown in Figure 3.

Further details about the training and testing variables can be found in the code repository linked in section VIII.

IV. RESULTS AND DISCUSSION

This section presents the evaluation of VoxNet and PointNet on the ModelNet10 dataset for 3D object classification, as well as the performance of a 3D autoencoder for object reconstruction. Classification accuracy, model complexity, and training efficiency are compared for both

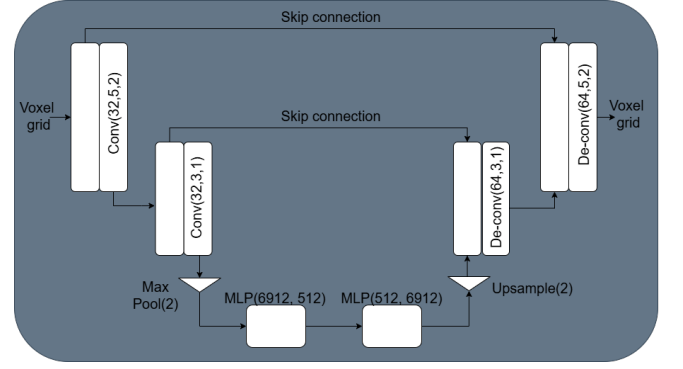


FIG. 3. **VoxNet reconstruction model** VoxNet autoencoder with skip connections. Denotation of the convolutional blocks are given in the caption to Figure 1. The encoder part is the VoxNet in its entirety, but with latent space of 512. The skip connections flow from the first and second encoder blocks, to the first and second decoder blocks respectively. Batch normalization was applied after each encoder block, and to the first and second decoder blocks.

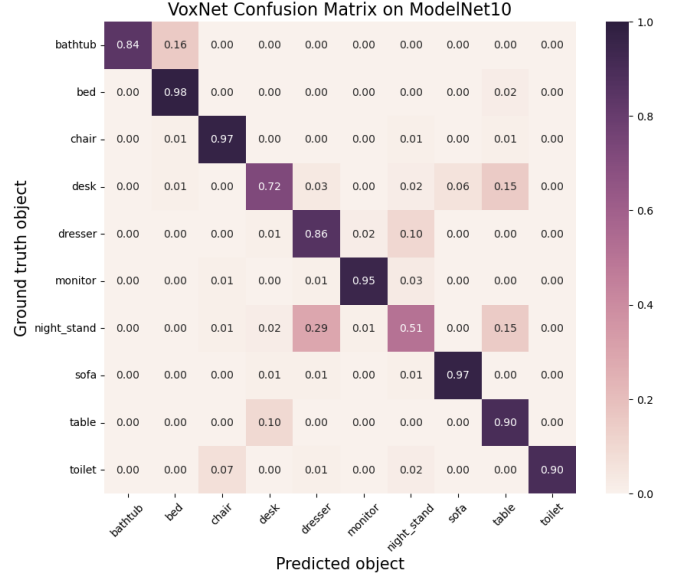


FIG. 4. **Confusion matrix of the VoxNet classifier.** Darker diagonal elements indicate higher accuracy for correctly predicted classes, while off-diagonal elements represent misclassifications and their respective proportions.

models, followed by an assessment of reconstruction quality. The performance of the classifiers was evaluated based on overall classification accuracy and detailed per-class results, visualized using confusion matrices.

VoxNet, which comprises approximately 918,000 trainable parameters, achieved an overall classification accuracy of 86.9%. The confusion matrix for VoxNet is shown in Figure 4.

As illustrated in Figure 4, VoxNet performs well across most categories, but shows difficulty in distinguishing between objects with similar geometric fea-

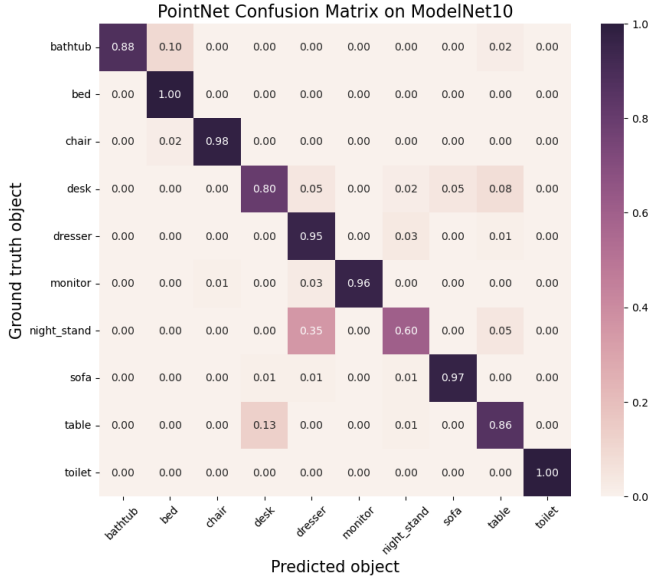


FIG. 5. **Confusion matrix of the PointNet classifier.** Diagonal elements indicate correct classifications, while off-diagonal elements represent misclassifications.

tures. Notably, nightstands are frequently confused with dressers and tables, and desks are sometimes misclassified as tables. Given the structural similarities between these objects, such confusion is understandable and highlights the limitations of voxel-based representations when fine-grained geometric detail is needed.

In comparison, PointNet, with approximately 803,000 trainable parameters, achieved a higher overall accuracy of 90.5%, as depicted in Figure 5.

The confusion matrix for PointNet indicates a more precise classification performance overall compared to VoxNet. However, the model still exhibits notable confusion between nightstands and dressers, even more so than VoxNet in some instances. This reinforces the challenge posed by classes with subtle geometric differences. Nonetheless, the improved performance of PointNet suggests that its point-based representation is more effective in capturing fine-grained spatial features essential for 3D object classification.

The original PointNet architecture also includes a mini-network called the T-Net, which learns affine transformations of the input space (3×3) and feature space (64×64). Its purpose is to align input data into a canonical form, improving robustness to geometric transformations such as rotation and scaling. However, when trained on the ModelNet10 dataset, incorporating the T-Net reduced classification accuracy to 88%. This decline is likely due to the fact that ModelNet10 contains pre-aligned, low-noise objects, making the learned transformations unnecessary. In this case, the T-Net adds extra complexity without providing a benefit. Conversely, in scenarios involving unaligned or variably scaled inputs, the T-Net would be beneficial, by aligning inputs before

they are processed by the main network, potentially improving accuracy [5].

Furthermore, PointNet has been shown to be more robust to missing or incomplete data than voxel based methods. As illustrated in Figure 8 of the PointNet paper, its classification accuracy degrades more slowly than VoxNet’s as the proportion of missing input increases. While VoxNet’s performance drops roughly linearly, PointNet maintains a relatively high accuracy up to a missing ratio of 0.75. Beyond that ratio, the accuracy of PointNet drops quickly, but still remains above that of VoxNet [5].

The initial autoencoder architecture, which consisted solely of the VoxNet as encoder, a decoder head and lacked skip connections, achieved an Intersection over Union (IoU) score of 56.2% over the entire test set. This relatively low score indicates that only about half of the reconstructed object overlapped with the ground truth, suggesting poor reconstruction quality. Visual inspection further confirmed that the output did not represent the original input with added shape completion. Instead, the network, often distorted the input, resulting in inconsistent outputs.

To address this, skip connections were introduced to pass spatial information from the encoder stages directly to the decoder. This allowed the model to more accurately reconstruct the observed parts of the object while completing the missing regions. With modification, the IoU improved significantly to 68.6%. As shown in Figure 6, the model successfully propagated the input structure to the output and filled in the missing parts of the object, yielding a close representation of the ground truth.

Since the network produces continuous output values in the range $[0, 1]$, a thresholding step is required to convert the output into a binary voxel representation. In this project, the same threshold was applied to all samples in the test set, which may have introduced bias. Optimizing the threshold individually per object and then computing the average IoU could provide a more accurate assessment of the model’s performance.

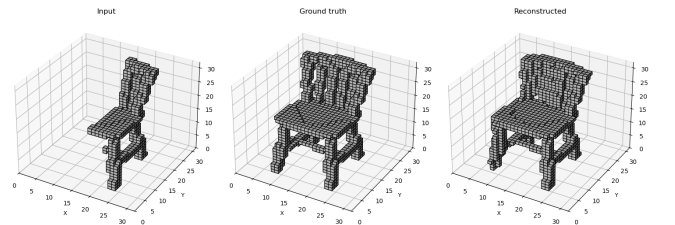


FIG. 6. **Illustration of the performance of the VoxNet autoencoder** The left-most plot show the input, its a chair split in half along the x axis, the middle plot is the ground truth, and the left-most plot show the reconstructed object.

V. CONCLUSIONS AND OUTLOOK

In conclusion, this project demonstrates that the first point cloud based model outperforms the first voxel based one, with PointNet having lower computational overhead and fewer parameters. Furthermore, the T-Net module within PointNet was found to reduce accuracy on a noise-free dataset. This suggests that its transformation learning may introduce unnecessary variance when input data is already well-aligned.

Additionally, I showed that a decoder head can be attached to an existing classifier to create an object reconstruction network, completing objects from partial input. These findings suggest potential for further optimization, particularly in tuning the output threshold. Either by finding a threshold for each object, automating it, or making it trainable. Moreover, it would be interesting to extend this model to a variational autoencoder, enabling

the generation of new objects from the latent space.

VI. CONTRIBUTIONS

The work was planned and performed by myself and I had no assistance from other students.

VII. CONFLICT OF INTEREST

I declare that there are no conflicts of interest related to this project. No financial, commercial, legal, or professional relationships influenced the work presented.

VIII. DATA AND CODE AVAILABILITY

The code is publicly available at: https://github.com/EricBlohm1/Project_3D_Deep_learning. More information is given in the README file.

-
- [1] P. Ambati, M. F. Hashmi, and A. Gupta, Deep learning frontiers in 3d object detection: A comprehensive review for autonomous driving, *IEEE Access* **12**, 173936 (2024), relevant to use in the introduction, to provide the reader with some context of why object classification is important.
 - [2] Y. Guo, H. Wang, Q. Hu, H. Liu, L. Liu, and M. Benamoun, Deep learning for 3d point clouds: A survey, *IEEE Transactions on Pattern Analysis and Machine Intelligence* **43**, 4338 (2021), relevant as related work. Discusses differences in many 3D neural network architectures.
 - [3] D. Selvaratnam and D. Bazazian, 3d reconstruction in robotics: A comprehensive review, *Computers & Graphics* **123**, 104256 (2025), important in the intro to state why 3D reconstruction is important.
 - [4] D. Maturana and S. Scherer, Voxnet: A 3d convolutional neural network for real-time object recognition, in *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (2015) pp. 922–928, this is one of the models I will implement. To compare this with PointNet.
 - [5] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, Pointnet: Deep learning on point sets for 3d classification and segmentation, in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2017) pp. 652–660, this is one of the models I will implement. To compare this with VoxNet.
 - [6] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, Pointnet++: Deep hierarchical feature learning on point sets in a metric space, *Advances in neural information processing systems* **30**, 1 (2017), relevant for related work and direct comparison. It improves on the PointNet architecture.
 - [7] Q.-C. Xu, T.-J. Mu, and Y.-L. Yang, A survey of deep learning-based 3d shape generation, *Computational Visual Media* **9**, 407 (2023), mentions various shape completion architectures relevant in the overview, and comparison of my choice.
 - [8] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, Dynamic graph cnn for learning on point clouds, *ACM Transactions on Graphics (TOG)* **38**, 1 (2019), relevant for related work. A bit complex for me to implement in the scope of this project, but interesting to discuss. Uses graph neural networks combined with PointNet.